

FRONTEND SOURCE CODE DOCUMENTATION

gudocs-fe

구독 관리 서비스 프론트엔드 기술 문서

FRAMEWORK

React 19

LANGUAGE

TypeScript 6

BUILD TOOL

Vite 8

상태 관리

Zustand 5

작성일: 2026년 05월 11일 | Repository: [scrumble-team/gudocs-fe](#)

목차 (Table of Contents)

1	프로젝트 개요	3
2	기술 스택 (Tech Stack)	4
3	디렉토리 구조	5
4	앱 진입점 & 라우팅	6
5	상태 관리 (Zustand)	8
6	API 레이어	9
7	TypeScript 타입 정의	10
8	커스텀 훅 (Custom Hooks)	11
9	컴포넌트 & 레이아웃	12
10	페이지 구성	13
11	설정 파일	15
12	데이터 흐름 & 아키텍처	16
13	개발 현황 & 향후 계획	17

서비스 소개

gudocs-fe는 사용자가 여러 구독 서비스(OTT, 음악 스트리밍, 클라우드 스토리지 등)를 하나의 인터페이스에서 통합 관리할 수 있는 **구독 관리 대시보드**의 프론트엔드입니다.

● 핵심 기능 목표

- › 구독 서비스 통합 관리
- › 월별 지출 현황 시각화
- › 결제 예정일 알림
- › 카테고리별 분석

● 개발 환경

- › Node.js (ESM 모듈)
- › 개발 서버: localhost:5173
- › 백엔드 API: localhost:8080
- › 패키지 매니저: npm

npm 스크립트

명령어	역할
npm run dev	Vite 개발 서버 시작 (HMR 포함, localhost:5173)
npm run build	TypeScript 타입 검사 후 Vite 프로덕션 빌드
npm run lint	ESLint 코드 정적 분석 실행
npm run preview	프로덕션 빌드 결과물 로컬 미리보기

핵심 의존성 (Dependencies)

분류	패키지	버전	역할
UI 프레임워크	react	^19.2.5	메인 UI 라이브러리 (최신 React 19)
UI 프레임워크	react-dom	^19.2.5	React DOM 렌더링
라우팅	react-router-dom	^7.14.2	SPA 클라이언트 라우팅
상태 관리	zustand	^5.0.12	경량 전역 상태 관리
HTTP 클라이언트	axios	^1.16.0	REST API 통신
UI 컴포넌트	@wanteddev/wds	^3.6.0	Wanted Design System 핵심
UI 컴포넌트	@wanteddev/wds-icon	^3.6.0	WDS 아이콘 라이브러리
UI 컴포넌트	@wanteddev/wds-lottie	^3.6.0	WDS Lottie 애니메이션
UI 컴포넌트	@wanteddev/wds-theme	^3.6.0	WDS 테마 토큰
UI 컴포넌트	@wanteddev/wds-engine	^3.6.0	WDS 엔진
스타일링	tailwindcss	^4.2.4	유틸리티 CSS 프레임워크
스타일링	@tailwindcss/vite	^4.2.4	Tailwind v4 Vite 플러그인
폰트	pretendard	^1.3.9	한국어 최적화 프리텐다드 폰트

개발 의존성 (devDependencies)

패키지	버전	역할
typescript	~6.0.2	TypeScript 컴파일러
vite	^8.0.10	빌드 도구 & 개발 서버
@vitejs/plugin-react	^6.0.1	Vite React 플러그인 (Fast Refresh)
eslint	^10.2.1	코드 정적 분석
typescript-eslint	^8.58.2	TypeScript ESLint 규칙

패키지	버전	역할
eslint-plugin-react-hooks	^7.1.1	React Hooks 규칙 검사
eslint-plugin-react-refresh	^0.5.2	React Refresh HMR 규칙
@types/react	^19.2.14	React 타입 정의
@types/node	^24.12.2	Node.js 타입 정의

Wanted Design System (WDS)란?

원티드(Wanted Lab)에서 만든 React 기반 디자인 시스템입니다. Button, TextField, Typography, Toast 등 일관된 UI 컴포넌트와 디자인 토큰(CSS 변수)을 제공하며, GitHub Package Registry를 통해 설치합니다.

```

gudocs-fe/ └─ public/ # 정적 에셋 (favicon, 아이콘 스프라이트 등) └─ favicon.svg └─
icons.svg └─ src/ # 메인 소스 코드 └─ api/ # API 통신 레이어 └─ instance.ts #
axios 인스턴스 설정 (baseUrl, timeout, cookies) └─ auth.ts # 인증 관련 API 함수
(login, signup, logout, getUser) └─ components/ # 공유 컴포넌트 └─
AuthProvider.tsx # 앱 마운트 시 사용자 정보 초기화 └─ PrivateRoute.tsx # 인증 필요 라우
트 보호 └─ hooks/ # 커스텀 React 훅 └─ useAuth.ts # 인증 비즈니스 로직 (로그인,
회원가입, 유효성 검사) └─ layouts/ # 레이아웃 컴포넌트 └─ Header.tsx # 공통 헤더
(로그, 사용자 드롭다운) └─ routes/ # 페이지 컴포넌트 (라우트별 분리) └─ login/ └─
LoginPage.tsx # 로그인 페이지 └─ register/ └─ RegisterPage.tsx # 회원가입
입 페이지 └─ dashboard/ └─ DashboardPage.tsx # 대시보드 메인 (개발 중) └─
mypage/ └─ MyPage.tsx # 마이페이지 (프로필, 비밀번호, 탈퇴) └─ stores/ #
Zustand 전역 상태 └─ useAuthStore.ts # 인증 상태 스토어 (user, isAuthenticated) └─
type/ # TypeScript 타입 정의 └─ api.ts # 공통 API 응답 타입 ApiResponse<T> └─
auth.ts # 인증 관련 타입 (User, LoginRequest, SignUpRequest) └─ App.tsx # 루트
컴포넌트 + 라우터 정의 └─ main.tsx # React 앱 진입점 (ThemeProvider 설정) └─ index.css
# 전역 스타일 (Tailwind, Pretendard 폰트) └─ index.html # HTML 진입점 └─
vite.config.ts # Vite 빌드 설정 └─ tsconfig.json # TypeScript 설정 (참조 루트) └─
tsconfig.app.json # 앱 TypeScript 설정 (ES2023, strict) └─ eslint.config.js # ESLint
flat config └─ Claude.md # 프로젝트 아키텍처 가이드 └─ design.md # WDS 스타일 가이드 └─
README.md # 프로젝트 소개 및 설치 안내

```

폴더 네이밍 규칙: 폴더명은 kebab-case, 파일명은 PascalCase(컴포넌트) 또는 camelCase(훅/유틸)를 사용합니다. 페이지는 routes/ 안에 도메인별로 분리합니다.

앱 부팅 순서



main.tsx — React 앱 마운트

```

// src/main.tsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "pretendard/dist/web/static/pretendard.css"; // 한국어 폰트
import "../index.css"; // Tailwind + 전역 스타일
import App from "../App.tsx";
import { ThemeProvider } from "@wanteddev/wds"; // WDS 테마 적용
import "@wanteddev/wds/global.css";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <ThemeProvider> { /* WDS 디자인 토큰 & 컴포넌트 활성화 */ }
    <App />
  </ThemeProvider>
</StrictMode>,
);

```

App.tsx — 라우트 정의

```

// src/App.tsx
function App() {
  return (
    <BrowserRouter>
      <AuthProvider> { /* 앱 시작 시 로그인 상태 복구 */ }
      <Routes>
        <Route path="/login" element={<LoginPage />} />
        <Route path="/register" element={<RegisterPage />} />
        <Route path="/dashboard/*" element={<DashboardPage />} />
        <Route path="/mypage" element={<MyPage />} />
        <Route path="*" element={<Navigate to="/dashboard" replace />} />
      </Routes>
    </AuthProvider>
  </BrowserRouter>
);
}

```

```
);  
}
```

라우트 구조 요약

경로	컴포넌트	설명	인증 필요
/login	LoginPage	로그인 폼	불필요
/register	RegisterPage	회원가입 폼	불필요
/dashboard/*	DashboardPage	메인 대시보드 (개발 중)	필요
/mypage	MyPage	프로필 관리	필요
*	Navigate	정의되지 않은 경로 → 대시보드로 리다이렉트	—

이 프로젝트는 **Zustand**를 사용해 전역 상태를 관리합니다. Redux보다 훨씬 적은 보일러플레이트로 동일한 기능을 구현할 수 있는 경량 상태 관리 라이브러리입니다.

useAuthStore — 인증 상태

```
// src/stores/useAuthStore.ts
import { create } from "zustand";
import type { User } from "../type/auth";

interface AuthStore {
  user: User | null;          // 로그인된 사용자 정보 (없으면 null)
  isAuthenticated: boolean; // 로그인 여부 플래그
  setUser: (user: User) => void; // 로그인 성공 시 호출
  logout: () => void;          // 로그아웃 시 호출
}

export const useAuthStore = create<AuthStore>((set) => ({
  user: null,
  isAuthenticated: false,
  setUser: (user) => set({ user, isAuthenticated: true }),
  logout: () => set({ user: null, isAuthenticated: false }),
}));
```

스토어 구조

항목	타입	설명
user	User null	현재 로그인된 사용자 객체 (id, email, name)
isAuthenticated	boolean	인증 여부 — PrivateRoute에서 사용
setUser(user)	Action	user 저장 + isAuthenticated = true 설정
logout()	Action	user = null + isAuthenticated = false 초기화

컴포넌트에서 사용하는 방법

```
// 전체 스토어 구독
const { user, isAuthenticated, setUser, logout } = useAuthStore();

// 특정 값만 구독 (성능 최적화 - 불필요한 리렌더링 방지)
```

```
const isAuthenticated = useAuthStore((state) => state.isAuthenticated);  
const user = useAuthStore((state) => state.user);
```

Zustand vs Redux: Zustand는 Provider 래핑 없이 바로 사용 가능하고, 보일러플레이트가 거의 없습니다. 현재 프로젝트처럼 인증 상태 등 단순한 전역 상태에 적합합니다.

axios 인스턴스 설정

```
// src/api/instance.ts
import axios from "axios";

export const server_axiosInstance = axios.create({
  baseURL: "http://localhost:8080", // Spring Boot 백엔드 서버
  timeout: 200000,                  // 200초 타임아웃
  withCredentials: true,            // 쿠키(세션) 자동 포함
});
```

● 설정 설명

- › **baseURL**: Spring Boot 백엔드 주소
- › **timeout**: 200초 (파일 업로드 등 대비)
- › **withCredentials**: HttpOnly 쿠키 기반 세션 인증

● 인증 방식

- › 쿠키 기반 세션 인증
- › 로그인 시 서버가 Set-Cookie 응답
- › 이후 요청에 자동으로 쿠키 전송
- › CORS 설정 필요 (백엔드)

인증 API 함수

```
// src/api/auth.ts
// 회원가입
export async function signUp(data: SignUpRequest): Promise<ApiResponse<SignUpResponseData>> {
  const response = await server_axiosInstance.post("/api/auth/signup", data);
  return response.data;
}

// 로그인
export async function login(data: LoginRequest) {
  const response = await server_axiosInstance.post("/api/auth/login", data);
  return response.data;
}

// 로그아웃
export async function logout() {
  const response = await server_axiosInstance.post("/api/auth/logout");
  return response.data;
}
```

```
// 현재 사용자 정보 조회 (세션 유지 확인)
export async function getUser(): Promise<ApiResponse<User>> {
  const response = await server_axiosInstance.get("/api/auth/me");
  return response.data;
}
```

API 엔드포인트 목록

Method	경로	함수명	설명
POST	/api/auth/signup	signUp	신규 사용자 등록
POST	/api/auth/login	login	이메일/비밀번호 로그인
POST	/api/auth/logout	logout	세션 종료
GET	/api/auth/me	getUser	현재 로그인 사용자 조회

공통 API 응답 타입 (src/type/api.ts)

```
// src/type/api.ts
export interface ApiResponse<T> {
  message: string; // 서버 응답 메시지
  success: boolean; // 성공 여부
  data: T; // 제네릭 - 실제 응답 데이터
}
```

제네릭 활용: ApiResponse<T>를 사용하면 모든 API의 응답 구조가 통일되면서도, 각 API마다 다른 데이터 타입을 타입 안전하게 사용할 수 있습니다. 예: ApiResponse<User>, ApiResponse<SignUpResponseData>

인증 관련 타입 (src/type/auth.ts)

```
// src/type/auth.ts

// 사용자 기본 정보
export interface User {
  id: number;
  email: string;
  name: string;
}

// 회원가입 요청 (User에서 id 제외 + 비밀번호 필드 추가)
export interface SignUpRequest extends Omit<User, "id"> {
  password: string;
  passwordConfirm: string;
}

// 회원가입 응답 데이터
export interface SignUpResponseData extends Omit<User, "password"> {
  name: string;
}

// 로그인 요청 (email + password만)
export interface LoginRequest extends Omit<User, "id" | "name"> {
  password: string;
}
```

타입 상속 구조

타입	베이스	필드
User	—	id, email, name
SignUpRequest	User (id 제외)	email, name, password, passwordConfirm
SignUpResponseData	User	id, email, name
LoginRequest	User (id, name 제외)	email, password

TypeScript Strict Mode 활성화: `tsconfig.app.json`에서 `strict` 옵션이 모두 켜져 있어 `any` 타입 사용 금지, 미사용 변수 오류, `nullish` 체크 등을 컴파일 타임에 강제합니다.

이 프로젝트는 비즈니스 로직을 컴포넌트에서 분리하기 위해 커스텀 훅 패턴을 사용합니다. 컴포넌트는 UI 렌더링만 담당하고, 로직은 훅에서 처리합니다.

useAuth — 인증 로직 훅 (src/hooks/useAuth.ts)

함수/값	타입	설명
validateEmail	(email: string) → boolean	이메일 형식 정규식 검증
validatePassword	(password: string) → boolean	비밀번호 최소 8자 검증
validatePasswordMatch	(pw, confirm) → boolean	비밀번호 일치 여부 검증
handleLogin	({ email, password }) → Promise	로그인 API 호출 → 스토어 업데이트 → 대시보드 이동
handleRegister	(SignUpRequest) → Promise	회원가입 API 호출 → 성공 토스트 → 로그인 이동
handleLogout	() → void	로그아웃 API 호출 → 스토어 초기화 → 로그인 이동

```
// 사용 예 (LoginPage.tsx)
const { handleLogin, validateEmail, validatePassword } = useAuth();

const onSubmit = async (e: FormEvent) => {
  e.preventDefault();
  if (!validateEmail(email)) { setEmailError("올바른 이메일 형식이 아닙니다."); return; }
  if (!validatePassword(password)) { setPasswordError("8자 이상 입력해주세요."); return; }
  await handleLogin({ email, password });
};
```

훅 분리의 장점: 같은 유효성 검사 로직을 LoginPage와 RegisterPage 양쪽에서 재사용합니다. 컴포넌트 파일이 UI 로직만 담아 가독성이 높아집니다.

AuthProvider (src/components/AuthProvider.tsx)

앱이 처음 로드될 때 서버에 GET /api/auth/me를 호출하여 세션이 유효한지 확인하고, Zustand 스토어의 인증 상태를 복구합니다.

```
// 앱 시작 시 자동으로 실행되는 인증 상태 복구 로직
useEffect(() => {
  const fetchUser = async () => {
    try {
      const { data } = await getUser(); // GET /api/auth/me
      setUser(data); // 세션 유효 → 스토어에 유저 저장
    } catch {
      logout(); // 세션 만료 → 인증 상태 초기화
    }
  };
  fetchUser();
}, []);
```

PrivateRoute (src/components/PrivateRoute.tsx)

인증이 필요한 페이지를 감싸는 래퍼입니다. 로그인되지 않은 상태에서 접근하면 자동으로 로그인 페이지로 이동합니다.

```
// 사용 예 (App.tsx에서 보호할 Route를 감쌀 때)
<Route path="/mypage" element={
  <PrivateRoute>
    <MyPage />
  </PrivateRoute>
} />
```

Header Layout (src/layouts/Header.tsx)

모든 페이지 상단에 표시되는 공통 헤더입니다. 인증 상태에 따라 다른 UI를 보여줍니다.

● 로그인 상태

- › G 로고 + "구독 관리" 텍스트 (대시보드 링크)
- › 사용자 이니셜 아바타 (보라 그라디언트)
- › 사용자 이름 표시
- › 드롭다운 메뉴 (마이페이지 / 로그아웃)

● 비로그인 상태

- › G 로고 + "구독 관리" 텍스트 (홈 링크)
- › "로그인" 버튼

LoginPage (src/routes/login/LoginPage.tsx)

항목	내용
레이아웃	화면 중앙 카드 형태
입력 필드	이메일, 비밀번호
유효성 검사	이메일 정규식, 비밀번호 8자 이상
에러 처리	WDS FormErrorMessage 컴포넌트로 필드 하단에 표시
성공 처리	/dashboard 로 이동
기타	이미 로그인된 경우 /dashboard 로 자동 리다이렉트, 회원가입 링크

RegisterPage (src/routes/register/RegisterPage.tsx)

항목	내용
레이아웃	화면 중앙 카드 형태
입력 필드	이름, 이메일, 비밀번호, 비밀번호 확인 (총 4개)
유효성 검사	이메일 형식, 비밀번호 8자 이상, 비밀번호 일치 여부
에러 처리	각 필드 하단에 인라인 에러 메시지
성공 처리	성공 토스트 알림 후 /login 이동
기타	이미 로그인된 경우 /dashboard 리다이렉트, 로그인 링크

DashboardPage (src/routes/dashboard/DashboardPage.tsx)

현재 개발 중(WIP)인 페이지입니다. Header + "대시보드" 제목만 표시되는 플레이스홀더 상태이며, 구독 목록, 월별 지출, 알림 기능은 향후 구현 예정입니다.

MyPage (src/routes/mypage/MyPage.tsx)

사용자 프로필 관리 페이지입니다. 세 가지 섹션으로 구성됩니다.

섹션	기능	구현 상태
프로필 카드	이니셜 아바타, 이름, 이메일, 인증 배지	완료
기본 정보	이름 수정 (저장/취소), 이메일 (읽기 전용)	완료
비밀번호 변경	현재 비밀번호, 새 비밀번호, 확인 입력	API 연동 대기
위험 영역	계정 삭제 (비밀번호 재확인 후 삭제)	API 연동 대기

vite.config.ts — 빌드 설정

```
// vite.config.ts
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react"; // React Fast Refresh (HMR)
import tailwindcss from "@tailwindcss/vite"; // Tailwind v4 (별도 postcss 불필요)

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

tsconfig.app.json — TypeScript 설정

옵션	값	의미
target	ES2023	최신 JS 문법 사용 가능
module	ESNext	ESM 모듈 시스템
jsx	react-jsx	자동 JSX Transform (import React 불필요)
moduleResolution	bundler	Vite 번들러 기반 모듈 해석
strict	true	엄격한 타입 검사 활성화
noUnusedLocals	true	미사용 변수 오류 처리
noUnusedParameters	true	미사용 파라미터 오류 처리
noFallthroughCasesInSwitch	true	switch 문 fallthrough 방지

eslint.config.js — 코드 품질

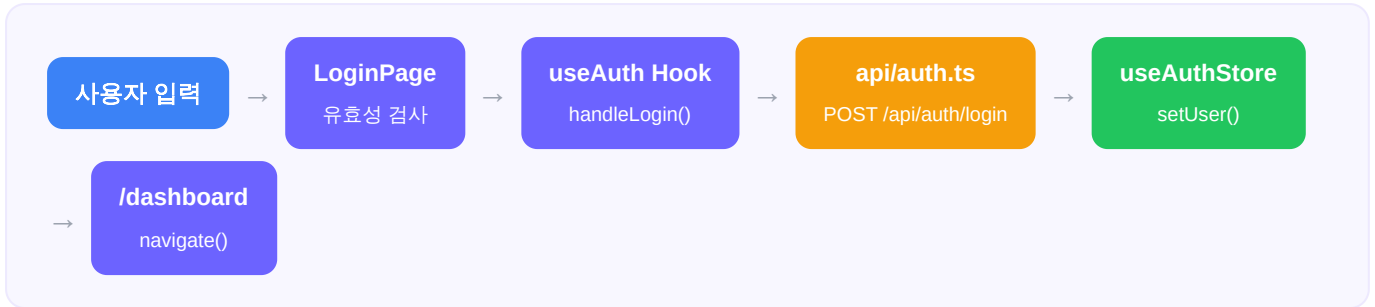
플러그인	역할
@eslint/js recommended	기본 JavaScript 규칙
typescript-eslint recommended	TypeScript 전용 규칙 (any 사용 경고 등)
eslint-plugin-react-hooks	useEffect 의존성 배열, Hook 호출 순서 등
eslint-plugin-react-refresh	Vite HMR과 호환되는 컴포넌트 구조 강제

```
/* src/index.css */
@import "tailwindcss"; /* Tailwind v4 유틸리티 클래스 활성화 */

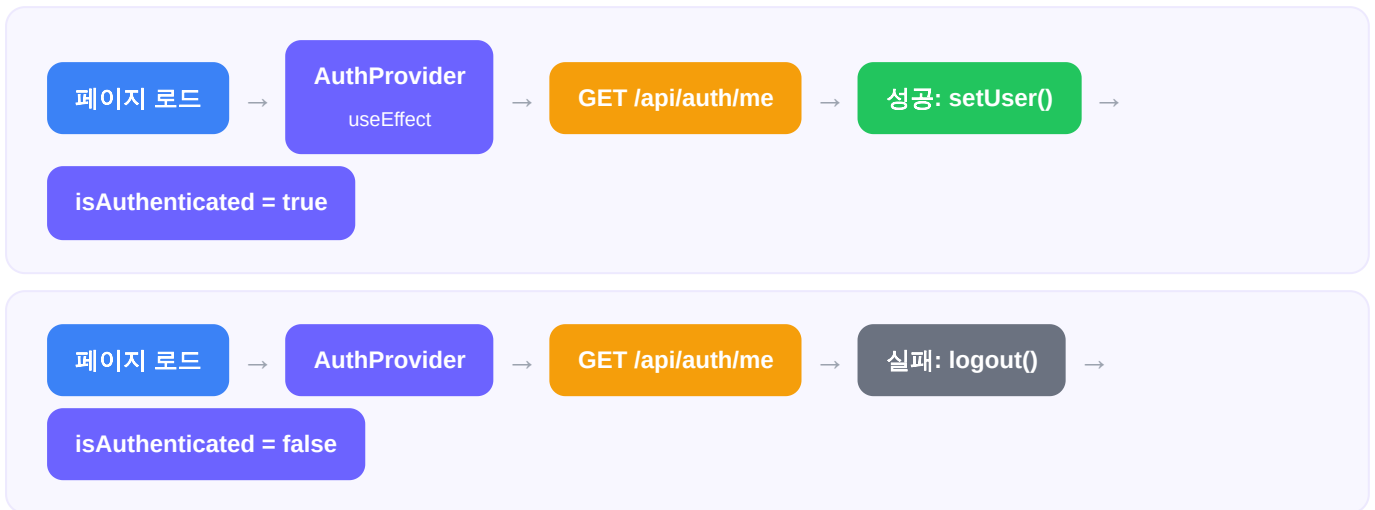
@theme {
  --font-sans: "Pretendard", sans-serif; /* Tailwind의 기본 sans 폰트 재정의 */
}

* {
  font-family: "Pretendard", sans-serif; /* 전체 요소에 Pretendard 폰트 적용 */
}
```

로그인 데이터 흐름



앱 초기화 흐름 (새로고침 시)



전체 아키텍처 계층 구조

계층	파일	역할
진입점	main.tsx, App.tsx	앱 부팅, ThemeProvider, BrowserRouter, 라우트 정의
페이지 (View)	routes/**/*Page.tsx	UI 렌더링, 사용자 입력 처리, 훅 호출
레이아웃	layouts/Header.tsx	공통 UI 구조
공유 컴포넌트	components/*.tsx	AuthProvider, PrivateRoute — 횡단 관심사
비즈니스 로직	hooks/useAuth.ts	API 호출 조율, 유효성 검사, 라우팅 결정
전역 상태	stores/useAuthStore.ts	Zustand — 앱 전체 공유 상태
API 레이어	api/instance.ts, api/auth.ts	axios 설정, HTTP 요청 함수

계층	파일	역할
타입 정의	type/api.ts, type/auth.ts	TypeScript 인터페이스

설계 원칙: 각 계층은 단일 책임을 가집니다. 페이지 컴포넌트는 UI만, 혹은 로직만, API 레이어는 HTTP 통신만 담당합니다. 이를 통해 유지보수성과 테스트 용이성을 높입니다.

구현 완료 기능

도메인	기능	상태
인증	이메일/비밀번호 로그인	완료
인증	회원가입 (이름, 이메일, 비밀번호)	완료
인증	로그아웃	완료
인증	세션 유지 (새로고침 후 로그인 상태 복구)	완료
마이페이지	사용자 정보 조회	완료
마이페이지	이름 수정	완료 (UI)
마이페이지	비밀번호 변경	API 연동 대기
마이페이지	계정 삭제	API 연동 대기

향후 구현 예정 기능 (Backlog)

도메인	기능
구독 관리	구독 목록 조회, 구독 상세, 구독 추가/수정/삭제, 일시정지/재개
대시보드	메인 화면 — 구독 카드 목록, 월별 지출 요약
지출 분석	월별 총액, 카테고리별 분류, 6개월 추이 차트, CSV 내보내기
알림	알림 설정, 미읽음 배지, 읽음 처리, 삭제

현재 프로젝트는 인증(Auth) 기반 기능 구현이 완료된 단계이며, 구독 관리 및 분석 기능은 백엔드 API 개발과 병행하여 순차적으로 추가될 예정입니다.

코딩 컨벤션 요약

● 필수 규칙

- › TypeScript strict 모드 필수
- › 함수형 컴포넌트만 사용
- › any 타입 사용 금지
- › 비즈니스 로직은 커스텀 훅으로 분리
- › 옵셔널 체이닝(?.) 활용

● 스타일 규칙

- › WDS 컴포넌트 우선 사용
- › 커스텀 CSS 최소화
- › Tailwind 유틸리티 클래스 활용
- › 토스트 알림은 WDS useToast 사용
- › 확인 모달은 WDS Modal 사용